Revisiting occlusion-aware association for zero-shot MASA tracking: A reproducibility study and a simple appearance–geometry fusion baseline

Group 2

Nguyễn Quốc Hùng, Trần Đại Phong Lâm, Huỳnh Phạm Gia Minh, Trương Nguyễn Triệu Vỹ

Abstract

MASA-OA reports that freezing the appearance memory of tracks during occlusion improves zero-shot multi-object tracking with MASA embeddings. We re-examine this claim. Under the original ground-truth-box protocol, the freeze is overwritten by an end-of-frame update in the released code; enforcing it as described lowers HOTA on MOT15-train from 96.96 to 90.41, the level of plain MASA (90.07), and a tracker without embeddings reaches 97.67, so the protocol cannot measure the contribution of appearance. We therefore build a controlled benchmark in which eight trackers share identical YOLOX-X detections, MASA-R50 embeddings and camera-motion estimates, with configurations locked on separate tuning splits before evaluation on MOT17-val, MOT20-val and DanceTrack-val. A training-free baseline, MASA-RF, which fuses MASA appearance with geometry in a three-stage cascade and keeps an always-updated memory, obtains the highest HOTA on all splits (68.97, 60.11, 56.79). In the pre-specified comparison with the strongest baseline, the gain is significant on DanceTrack-val (+2.37, 95% CI [0.32, 4.59]) but not on MOT17-val or MOT20-val; exact per-sequence tests with Holm correction confirm it against three of seven baselines on DanceTrack-val. Freezing also lowers HOTA with real detections: a hard occlusion freeze costs 0.99–3.62 in MASA-RF, and enforcing the MASA-OA freeze costs 0.43–3.32 in the authors' code. Association should therefore be evaluated with real detections, and the appearance memory should keep updating.

Keywords: multi-object tracking, data association, zero-shot appearance embedding, occlusion, appearance memory, reproducibility

1. Introduction

Multi-object tracking (MOT) assigns a consistent identity to every object of interest across the frames of a video. The dominant paradigm, tracking-by-detection, runs a detector on each frame and links the detections to existing tracks by solving an assignment problem [1] over a cost that combines motion and appearance cues [2–4]. Motion cues, typically a Kalman prediction compared to the detection by intersection-over-union (IoU), are reliable when objects move smoothly and stay well separated. Appearance cues, obtained from a re-identification (ReID) embedding [5], become necessary when objects cross, are occluded, or move non-linearly [6].

Recent foundation models offer appearance embeddings that need no tracking-specific training. MASA [7] learns instance-level correspondences from Segment Anything [8] masks and can be attached to any detector as a zero-shot association head. Its tracker, inherited from quasi-dense similarity learning [9, 10], matches detections to tracks mainly by appearance, within a spatial gate, and updates each track's embedding with an exponential moving average (EMA). When two people overlap, however, the crop of one contains part of the other, and the updated embedding may drift toward the wrong identity. MASA-OA [11] addresses this by clustering overlapping detections, matching them with Kalman–IoU and freezing their appearance memory. The authors report higher HOTA than MASA on MOT15, MOT16, MOT17 and CTMC-v1 and comparable HOTA on MOT20, with ground-truth boxes as input.

While attempting to extend MASA-OA, we found three problems with the evidence for this conclusion. First, in the released code an end-of-frame update rewrites the memory of every matched track with the

default EMA rate, so the freeze has no effect. Second, with ground-truth boxes, a tracker that uses no embedding outperforms MASA-OA, so the protocol cannot isolate the contribution of appearance. Third, MASA-OA is compared only with MASA and its own variants, not with standard trackers. These findings change the question: not *how to protect the appearance memory*, but *whether protection helps at all, and how zero-shot appearance should be combined with geometry when real detections are used.* Similar re-examinations in metric learning have shown that reported progress can shrink considerably under a consistent evaluation protocol [12].

This paper makes the following contributions:

1. A reproducibility study of MASA-OA under its own protocol, with code-level ablations that separate the effect of each mechanism (Section 4).
2. A controlled detector-based benchmark of eight trackers that share detections, embeddings and camera motion, in which baselines run from official code and configurations are locked on tuning splits before evaluation (Section 6).
3. MASA-RF (MASA with Robust Fusion), a training-free baseline that fuses MASA appearance with IoU and centre geometry in a three-stage cascade and keeps a slow, always-updated appearance memory (Section 5).
4. A statistical analysis that separates the pre-specified primary comparison from exploratory comparisons, and tests differences per sequence with exact permutation tests and multiplicity correction (Section 7.2).
5. Direct evidence on memory protection with real detections: an appearance-memory sweep, a hard occlusion freeze, and the MASA-OA mechanism ablation repeated on detector output (Sections 7.5 and 7.6).

The remainder of this paper is organised as follows. Section 2 reviews related work and Section 3 gives the necessary background. Section 4 reports the reproducibility study, Section 5 describes MASA-RF, and Section 6 the experimental setup. Section 7 presents the results, Section 8 discusses them together with threats to validity, and Section 9 concludes.

2. Related work

Motion-based tracking-by-detection. Tracktor [13] and CenterTrack [14] showed that strong detectors make simple frame-to-frame association competitive. ByteTrack [2] associates low-confidence detections in a second stage instead of discarding them, which recovers partially occluded objects. OC-SORT [3] corrects the Kalman state after occlusion and adds a velocity-direction consistency term. Hybrid-SORT [15] adds weak cues such as confidence and height, UCMCTrack [16] models motion on the ground plane with uniform camera-motion compensation, and MotionTrack [17] and DiffMOT [18] learn motion models for crowded scenes and non-linear motion. Camera motion is commonly estimated from sparse feature tracks [19, 20] with a robust fit [21].

Appearance cues and appearance memory. Joint detection-and-embedding models (JDE [22], FairMOT [23]) learn ReID features together with detection; a survey of person ReID is given in [5]. StrongSORT [4] combines a stronger ReID model with an EMA feature bank, camera-motion compensation and appearance-free link refinement. GHOST [24] shows that simple combinations of appearance and motion, with careful handling of short- and long-term cues, are strong. SMILEtrack [25] learns a similarity module for occlusion-aware matching. QDTrack [9, 10] learns dense similarities and matches with a bi-directional softmax, and TETA [26] extends class-agnostic association to many categories. MASA [7] transfers this matching to arbitrary detectors using SAM [8] supervision; the variant used here has a ResNet-50 [27] backbone. MASA-OA [11] adds occlusion-aware matching and memory freezing. Our work studies the same design space but questions whether memory protection is the source of improvement.

Learned and end-to-end association. Graph-based solvers learn association offline [28, 29], while transformer trackers propagate track queries across frames [30–32], in some cases with an explicit long-term memory [33]. These methods require tracking-specific training and are outside the training-free setting considered here.

Evaluation and reproducibility. MOTChallenge [34] introduced the MOT15–MOT17 benchmarks with the CLEAR-MOT and IDF1 metrics; its platform also hosts the crowded MOT20 benchmark.[1] HOTA [35] balances detection and association accuracy and is our primary metric. DanceTrack [6] contains dancers with uniform appearance and diverse motion, a setting in which association is the main difficulty. Pineau et al. [36] argue for released code, fixed protocols and reported variance, and Demšar [37] recommends paired, non-parametric comparisons over data sets rather than single aggregate scores. We follow both recommendations.

3. Background

3.1. Problem formulation

At frame t a detector returns boxes b_i with scores s_i, and an embedding network returns a vector $\mathbf{e}_i$ for each box. A track j keeps a Kalman state, from which a predicted box $\hat{b}_j$ is derived, and an appearance memory $\mathbf{e}_j$. Association selects a one-to-one matching between detections and tracks that minimises a cost C_{ij}, subject to an acceptance threshold. After a match, the memory is updated as

$$\mathbf{e}_j \leftarrow (1-\alpha)\,\mathbf{e}_j + \alpha\,\mathbf{e}_i, \tag{1}$$

where α is the weight of the new sample: $\alpha = 0$ freezes the memory, and a small α makes it change slowly.

3.2. Evaluation metrics

At a localisation threshold β, predictions and ground truth are matched into true positives TP_β, false negatives FN_β and false positives FP_β. HOTA [35] combines detection accuracy and association accuracy:

$$\mathrm{DetA}_\beta = \frac{|\mathrm{TP}_\beta|}{|\mathrm{TP}_\beta| + |\mathrm{FN}_\beta| + |\mathrm{FP}_\beta|}, \qquad \mathrm{AssA}_\beta = \frac{1}{|\mathrm{TP}_\beta|} \sum_{c \in \mathrm{TP}_\beta} \frac{|\mathrm{TPA}(c)|}{|\mathrm{TPA}(c)| + |\mathrm{FNA}(c)| + |\mathrm{FPA}(c)|}, \tag{2}$$

$$\mathrm{HOTA}_\beta = \sqrt{\mathrm{DetA}_\beta \cdot \mathrm{AssA}_\beta}, \qquad \mathrm{HOTA} = \frac{1}{19} \sum_{\beta \in \{0.05, 0.10, \ldots, 0.95\}} \mathrm{HOTA}_\beta. \tag{3}$$

We also report IDF1, which measures identity preservation globally [34], and the number of identity switches (IDSW).

3.3. MASA-OA

MASA-OA [11] partitions the detections of a frame by overlap. Non-overlapping detections are matched by appearance and update the memory with $\alpha = 0.8$. Overlapping detections are matched by IoU with the Kalman prediction, and according to the paper their embeddings are frozen ($\alpha = 0$). Remaining detections are matched in a final Kalman–IoU pass with $\alpha = 0.35$. The paper's key claim is that this freeze prevents contamination of the appearance memory during occlusion.

4. Reproducibility study of MASA-OA

All experiments in this section follow the MASA-OA protocol: ground-truth boxes as input, MASA-R50 embeddings extracted from those boxes, and the official TrackEval implementation of the metrics on MOT15-train (11 videos, 5,500 frames, 39,905 valid boxes).

[1] https://motchallenge.net

Table 1: Mechanism ablation of the released MASA-OA tracker on MOT15-train with ground-truth boxes. Each row patches a copy of the author file; the original file is never modified.

Variant of the author code	HOTA	AssA	IDSW	IDF1
Released code (freeze overwritten)	96.96	94.12	101	95.62
Kalman correction applied once	96.57	93.51	118	95.22
Freeze enforced as described ($\alpha = 0$)	90.41	81.90	196	87.26
Freeze enforced + Kalman once	90.15	81.55	211	87.01
Overlap threshold IoU ≥ 0.3 as described	94.94	90.25	112	93.03
Both points as described	88.42	78.66	175	85.91
MASA (published result file)	90.07	81.30	271	86.74

Table 2: Ground-truth-box protocol, HOTA / IDSW. "Geometry only" sets the appearance weight to zero.

Configuration	MOT15-train	MOT17-04 + MOT17-13
MASA-OA (released code)	96.96 / 101	98.96 / 53
Geometry only (Kalman + IoU + centre)	97.67 / 238	99.13 / 238
Kalman + IoU only	95.48 / 642	98.08 / 753
Appearance only (MASA)	92.03 / 213	99.51 / 13
Appearance + geometry	98.11 / 51	99.57 / 11

The freeze is inert in the released code. The released tracker[2] updates overlapping tracks with $\alpha = 0$ (line 530) and the remaining tracks with $\alpha = 0.35$ (line 549). At the end of the frame (line 559), however, it calls the update again for *every* matched track with the default $\alpha = 0.8$. By Eq. (1), the second call yields the same memory as no freeze at all; it also applies a second Kalman correction with the same measurement. Table 1 patches copies of the author file to isolate each mechanism. Enforcing the freeze as described removes the whole improvement (90.41, versus 90.07 for MASA), whereas keeping the Kalman–IoU matching of overlapping boxes without the freeze retains it (96.96 and 96.57).

The ground-truth-box protocol saturates. Table 2 compares the released MASA-OA tracker with our association code run with individual cues switched off. With perfect boxes, geometry alone (Kalman prediction, IoU and a normalised centre distance) reaches 97.67 HOTA on MOT15 and outperforms MASA-OA; on MOT17-04 and MOT17-13 every configuration lies between 98.1 and 99.6. Appearance-only matching is the weakest configuration on MOT15. Hence the protocol cannot measure whether a change to the appearance pathway helps.

Further observations. Three further points concern the reported evidence. (i) Eq. (3) of the paper defines HOTA as the geometric mean of localisation accuracy and association accuracy, whereas HOTA uses detection accuracy (Eq. (3) above). (ii) When the published MASA-OA result file for MOT15 is scored with TrackEval, it contains 586 false positives and no false negatives (DetA 98.45, MOTA 98.31), which indicates that its input was not identical to the evaluated ground-truth set. (iii) On MOT20 the paper reports lower HOTA for MASA-OA than for MASA (98.1 versus 98.8) together with higher AssA, AssRe and AssPr, which by the definition of HOTA implies a change in detection accuracy under nominally fixed boxes.

An initial memory-gating proposal fails. Our initial plan generalised the freeze: an embedding contamination risk (ECR), computed from the deviation from the track history, similarity to neighbouring tracks, box overlap and Kalman residual, sets a reliability ρ that (a) weights appearance against geometry in the

[2] https://github.com/gonzalocarreteroh/MASA-OA, commit 2dc7305, file masa/models/tracker/masa_tao_tracker.py.

matching cost and (b) gates the memory update. On MOT15-train, reliability-weighted matching alone gave 97.34 HOTA (+0.38 over the released code; 95% CI $[-0.48, +1.13]$, paired bootstrap over videos), and adding the memory gate to it lowered HOTA to 94.37 (-2.97). Applied to MASA-OA's matching, the gate gave 92.80: better than a hard freeze (90.35 in our re-implementation) but worse than the released code, whose freeze is inert (96.96). Because geometry alone already reaches 97.67, this protocol cannot test the reliability hypothesis, so we moved to real detections.

5. MASA-RF: a simple fusion baseline

MASA-RF is a training-free tracker that follows the ByteTrack/BoT-SORT track life cycle (high- and low-score detections, unconfirmed tracks, a lost-track buffer) and changes the cost used for high-score detections and the appearance memory. Fig. 1 and Algorithm 1 give the complete procedure for one frame.

Fused cost. For a detection i and a track j, the cost used in stages 1 and 3 is

$$C_{ij} = 1 - s_i \Big[w'_{ij}\, A_{ij} + (1 - w'_{ij})\, G_{ij} \Big], \qquad G_{ij} = (1 - c)\,\mathrm{IoU}(b_i, \hat{b}_j) + c\, S_{ij}, \tag{4}$$

where A_{ij} is the MASA appearance score, the mean of the cosine similarity and the bi-directional softmax similarity used by MASA [7, 9]; $S_{ij} = \exp\big(-2\|(\mathbf{p}_i - \hat{\mathbf{p}}_j) \oslash \hat{\mathbf{d}}_j\|^2\big)$ is a centre-proximity score with box centres $\mathbf{p}$ normalised by the predicted box size $\hat{\mathbf{d}}$; and multiplying by s_i follows the score fusion of ByteTrack [2]. Pairs with G_{ij} below a threshold are invalid, and an optional cosine gate invalidates pairs with cosine < 0.5 unless IoU ≥ 0.5. Assign removes pairs with cost above its threshold and then solves the Hungarian assignment [1], so that an invalid pair cannot displace a valid one.

Geometry-ambiguity weighting (GAW). The appearance weight starts from a constant w and grows when geometry cannot separate candidates. With m_{ij} the smaller of the row and column margins between the best and second-best values of G,

$$w'_{ij} = w + (1 - w)\,\kappa\,\big(1 - \min(m_{ij}/0.3,\ 1)\big), \qquad \kappa \in [0, 1]. \tag{5}$$

Appearance memory. Tracks matched to a high-score detection (stages 1 and 3) always update their embedding by Eq. (1) with a slow rate, $\alpha = 0.1$, instead of the $\alpha = 0.8$ of MASA and MASA-OA. Low-score detections never write to the memory. An optional occlusion-risk memory (ORM) scales α by $\max(0.1,\ 1 - 0.9\, r_i)$, where the occlusion risk r_i is the largest fraction of box i covered by another detection.

Complexity. For n detections and tracks, building the cost matrices takes $O(n^2)$ operations, plus $O(n^2 \log n)$ when GAW sorts the margins, and the Hungarian assignment takes $O(n^3)$ in the worst case. Section 7.7 reports measured run time.

Locked configurations. Table 3 lists the configurations selected on the tuning splits (Section 6). On the MOT family, the search selected GAW and the cosine gate and no centre term; on DanceTrack, it selected the centre term and ORM. Both searches selected the slowest memory rate that was offered ($\alpha = 0.1$).

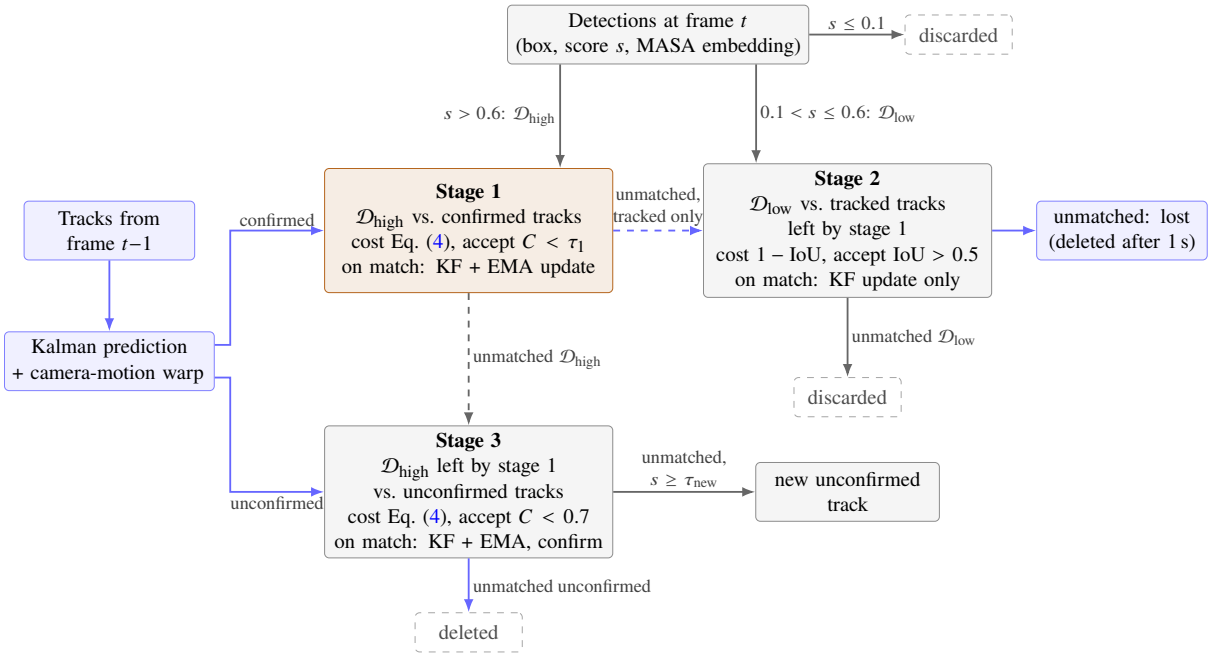


Figure 1: MASA-RF association for one frame, as implemented. Detections (grey) flow from top to bottom and are split by score; tracks (blue) flow from left to right after a Kalman prediction warped by the estimated camera motion. Confirmed tracks include tracked and lost tracks; unconfirmed tracks were created at the previous frame. Dashed arrows carry what stage 1 leaves unmatched: tracked (not lost) tracks go to stage 2, and high-score detections go to stage 3, bypassing stage 2. KF: Kalman filter.

Table 3: MASA-RF configurations locked on the tuning splits before evaluation.

Benchmark family	w	c	GAW κ	Cosine gate	ORM	α	τ_1	τ_{new}	Min. G_{ij}
MOT17 / MOT20	0.3	0	1	0.5	off	0.1	0.7	0.6	0.15
DanceTrack	0.3	0.25	0	off	on	0.1	0.9	0.7	0.05

Algorithm 1 MASA-RF association at frame t

Input: detections $\{(b_i, s_i, \mathbf{e}_i)\}$, tracks $\mathcal{T}$, camera transform H_t
Output: confirmed tracks matched at frame t

Discard detections with $s_i \le 0.1$; $\mathcal{D}_{\text{high}} \leftarrow \{i : s_i > 0.6\}$; $\mathcal{D}_{\text{low}} \leftarrow$ the remaining detections
for all tracks $j \in \mathcal{T}$ **do** predict the Kalman state and warp it by H_t
end for
$\mathcal{T}_c \leftarrow$ confirmed tracks (tracked or lost); $\mathcal{T}_u \leftarrow$ unconfirmed tracks
$M_1 \leftarrow \textsc{Assign}(C(\mathcal{D}_{\text{high}}, \mathcal{T}_c), \tau_1)$ ▷ Eq. (4)
Update the Kalman state and the memory (Eq. (1)) of the tracks in M_1
$\mathcal{T}_r \leftarrow$ tracks of $\mathcal{T}_c$ not in M_1 whose state is *tracked*
$M_2 \leftarrow \textsc{Assign}(1 - \text{IoU}(\mathcal{D}_{\text{low}}, \mathcal{T}_r), 0.5)$; update only the Kalman state of the tracks in M_2
$\mathcal{D}_r \leftarrow$ detections of $\mathcal{D}_{\text{high}}$ not in M_1
$M_3 \leftarrow \textsc{Assign}(C(\mathcal{D}_r, \mathcal{T}_u), 0.7)$; update Kalman state and memory; mark these tracks confirmed
Delete tracks of $\mathcal{T}_u$ not in M_3; mark tracks of $\mathcal{T}_c$ not in $M_1 \cup M_2$ as lost
Delete tracks that have been lost for more than 1 s
Create an unconfirmed track for every $i \in \mathcal{D}_r \setminus M_3$ with $s_i \ge \tau_{\text{new}}$

6. Experimental setup

Datasets. Table 4 summarises the reporting splits. MOT17-val is the second half of each MOT17 training video [2, 34]; 17.7% of its ground-truth boxes extend beyond the image, and four of seven videos have a moving camera. MOT20-val uses the four MOT20 training videos, since MOT20 has no validation split; it contains dense crowds with static cameras. DanceTrack-val [6] contains 25 videos of dancers with uniform appearance and non-linear motion. Tuning uses disjoint data: the first half of each MOT17 training video for the MOT family (MOT17 and MOT20), and six DanceTrack training videos for DanceTrack. No MOT20 data are used for tuning.

Shared inputs. All trackers receive identical inputs. Detections come from the YOLOX-X[3] checkpoints released with ByteTrack [2] (the model trained on CrowdHuman and the MOT17 train-half for MOT17; the full MOT17 model at 800×1440 for MOT20) and with DanceTrack [6]. Appearance embeddings (256-D) come from the released MASA-R50 model [7] with the detector boxes as regions of interest. Camera motion is a 2×3 similarity transform (rotation, uniform scale, translation) estimated from sparse optical flow [19, 20] with RANSAC [21]. Detection boxes are kept unclipped on MOT17 and clipped to the image on MOT20 and DanceTrack, matching the convention of each benchmark's ground truth. No model is trained or fine-tuned.

Compared methods. Table 5 lists the seven baselines and their components. We run the official code, unmodified on disk, of ByteTrack [2], OC-SORT [3], BoT-SORT[4] without ReID and with MASA embeddings as ReID features, Deep OC-SORT[5] with MASA embeddings, and the released MASA-OA tracker [11]. MASA association [7] is re-implemented because its released tracker is coupled to its own detector. Only the inputs are substituted in memory: cached detections, MASA embeddings instead of the original ReID networks, and cached camera motion instead of the original module. As a fidelity check, ByteTrack in our pipeline with the parameters of its official script reaches 67.92 HOTA on MOT17-val, against 67.77 when the official script (detector and tracker) is run end to end on the same frames and its output is scored with the same TrackEval code.

Protocol. The protocol and the search spaces were written before any validation result was computed (apart from a one-video smoke test of the adapters). Each method's hyper-parameters are searched on the tuning split and locked. Small search spaces are searched exhaustively (16–24 configurations for ByteTrack, OC-SORT, BoT-SORT, MASA and MASA-OA); larger ones use at most 48 random configurations per round. The three appearance-based trackers with large spaces received further rounds, for totals of 96 (MASA-RF), 102 (BoT-SORT with MASA ReID) and 132 (Deep OC-SORT) configurations per tuning split. Every method is then evaluated once on each reporting split. Appendix A lists the search spaces and locked configurations. A first run of this protocol cached detector boxes clipped to the image, which lowered all MOT17 results; after the fix, all tuning and evaluation were rerun with the same scripts, and the locked configurations of all methods are those produced by that rerun. Experiments that were designed after seeing the validation results are labelled *post hoc*; they do not change any locked configuration.

Metrics and statistical analysis. We report HOTA (primary), AssA, IDF1 and IDSW from the official TrackEval[6] implementation [35], on raw output without interpolation. The pre-specified primary analysis is a paired bootstrap [38] over sequences (2,000 resamples; HOTA is recomputed on each resample; 95% percentile interval) between MASA-RF and the strongest baseline of each split. We add two kinds of secondary analysis. The bootstrap is repeated against all baselines without multiplicity correction. Because a bootstrap over few sequences is coarse (MOT20-val has only 35 distinct resamples), and in the

[3] https://github.com/Megvii-BaseDetection/YOLOX

[4] Official implementation: https://github.com/NirAharon/BoT-SORT, commit 2519854.

[5] Official implementation: https://github.com/GerardMaggiolino/Deep-OC-SORT, commit 6bb51d0; we use MASA embeddings and set its `grid_off` option, so one embedding is computed per box.

[6] https://github.com/JonathonLuiten/TrackEval, commit 12c8791.

Table 4: Reporting splits. Moving camera: videos whose median estimated camera shift exceeds 1 pixel per frame.

Split	Videos	Frames	GT boxes	Boxes / frame	Moving camera
MOT17-val	7	2,652	53,890	20.3	4 / 7
MOT20-val	4	8,931	1,134,614	127.0	0 / 4
DanceTrack-val	25	25,508	225,148	8.8	15 / 25

Table 5: Components of the compared trackers. α: weight of the new sample in the appearance memory; for Deep OC-SORT it depends on the detection score, with an upper bound of 0.2 (MOT family) or 0.05 (DanceTrack).

Method	Motion model	First association	Second association	α
ByteTrack	Kalman	IoU	IoU, low-score detections	–
OC-SORT	Kalman, re-update	IoU, direction	IoU, last observation	–
BoT-SORT	Kalman, camera	IoU	IoU, low-score detections	–
BoT-SORT + MASA ReID	Kalman, camera	min(IoU, appearance)	IoU, low-score detections	0.1
Deep OC-SORT + MASA	Kalman, camera, re-update	IoU, direction, appearance	IoU, last observation	$\leq$0.2
MASA	none	appearance	none	0.8
MASA-OA	Kalman (overlaps only)	appearance; overlaps: IoU	IoU, remaining detections	0.8
MASA-RF	Kalman, camera	appearance + geometry	IoU, low-score detections	0.1

spirit of the paired non-parametric comparisons recommended by Demšar [37], we also apply an exact two-sided sign-flip permutation test to the per-sequence HOTA differences, with Holm correction [39] over the seven baselines of each split. With n sequences, the smallest attainable unadjusted p-value of this test is 2^{1-n}: 0.125 for MOT20-val and 0.016 for MOT17-val. After Holm correction, neither split can therefore reach $p < 0.05$, regardless of effect size, and only DanceTrack-val can support a confirmatory per-sequence claim.

Implementation and hardware. The pipeline is implemented in Python and evaluated on an AMD Ryzen 7 9800X3D CPU (8 cores) and an NVIDIA GeForce RTX 5070 Ti. Run times are measured one process at a time on an otherwise idle machine, with single-threaded numerical libraries for association, as detailed in Section 7.7.

7. Results

7.1. Main comparison

Table 6 reports all methods on the three reporting splits with their locked configurations. MASA-RF obtains the highest HOTA and AssA on all three. On DanceTrack-val it improves HOTA by 2.37 and IDF1 by 4.23 over the strongest baseline in each metric and has the fewest identity switches. On MOT17-val and MOT20-val its HOTA margin over the strongest baseline is below 0.6, other trackers have fewer identity switches, and MASA-OA has the highest IDF1 on MOT20-val. The released MASA-OA tracker is below MASA on MOT17-val and DanceTrack-val, and above it on MOT20-val. Appendix B gives the complete metrics and per-sequence results.

Table 6: Results on the reporting splits with locked configurations (raw output, no interpolation). All methods share detections, embeddings and camera motion. Best value per column in bold.

	MOT17-val				MOT20-val				DanceTrack-val			
Method	HOTA	AssA	IDF1	IDSW	HOTA	AssA	IDF1	IDSW	HOTA	AssA	IDF1	IDSW
ByteTrack [2]	67.20	68.67	79.18	237	58.80	54.44	73.83	2639	46.42	30.52	50.86	2084
OC-SORT [3]	66.82	69.87	78.29	226	54.99	53.61	70.64	**2065**	51.79	34.50	51.45	2159
BoT-SORT	68.69	70.81	80.87	200	59.52	55.60	74.22	3442	53.33	36.45	54.06	1931
BoT-SORT + MASA ReID	68.73	70.71	81.50	175	58.70	54.64	73.24	2605	54.01	37.18	54.02	2217
Deep OC-SORT + MASA	68.46	71.33	80.90	**158**	57.52	55.99	73.89	2347	54.42	37.39	53.40	2553
MASA [7]	66.29	66.51	76.85	275	58.13	54.88	73.10	2832	52.96	35.59	50.45	2319
MASA-OA [11]	65.54	66.69	75.48	300	59.51	56.18	**75.16**	2106	48.36	30.24	44.42	3028
MASA-RF (ours)	**68.97**	**71.49**	**81.69**	184	**60.11**	**56.51**	74.93	2282	**56.79**	**40.89**	**58.29**	**1710**

7.2. Statistical analysis

Primary analysis. The pre-specified comparison is with the strongest baseline of each split, marked † in Table 7. The gain is significant on DanceTrack-val (+2.37 against Deep OC-SORT, 95% CI [+0.32, +4.59]) and not on MOT17-val (+0.24 against BoT-SORT with MASA ReID, [−2.19, +1.29]) or MOT20-val (+0.59 against BoT-SORT, [−0.81, +1.29]).

Secondary analyses. Without multiplicity correction, the bootstrap interval excludes zero for all seven baselines on DanceTrack-val, five on MOT20-val and four on MOT17-val (Table 7). These counts should not be read as confirmatory: there are 21 uncorrected comparisons, and MOT20-val has only 35 distinct bootstrap resamples. Table 8 gives the more conservative per-sequence test. After Holm correction, the difference on DanceTrack-val is significant at the 0.05 level against ByteTrack, OC-SORT and MASA-OA, and is close to that level ($p = 0.067$) against BoT-SORT, BoT-SORT with MASA ReID, Deep OC-SORT and MASA. On MOT17-val and MOT20-val no comparison can reach this level by construction (Section 6), so win/loss counts are the informative quantity: on MOT17-val, MASA-RF beats BoT-SORT in 4 of 7 sequences and loses to BoT-SORT with MASA ReID in 5 of 7, which agrees with the small combined margins. Fig. 2 shows that on DanceTrack-val MASA-RF is better than Deep OC-SORT in 15 of 25 videos and better than BoT-SORT with MASA ReID in 17 of 25, so the gain is spread over many videos but is not uniform.

In summary, the evidence supports a real gain over ByteTrack, OC-SORT and MASA-OA on DanceTrack-val, supports a likely but not conclusively confirmed gain over the strongest appearance-based baselines on that split, and shows parity with the strongest baselines on MOT17-val and MOT20-val.

Table 7: HOTA(MASA-RF) − HOTA(baseline) on the combined split, with 95% paired-bootstrap interval over sequences. Bold: interval excludes zero, without multiplicity correction. †: strongest baseline of the split (pre-specified primary comparison).

Baseline	MOT17-val	MOT20-val	DanceTrack-val
ByteTrack	**+1.76** [+1.15, +3.40]	+1.31 [−0.30, +1.90]	**+10.37** [+7.77, +13.16]
OC-SORT	**+2.15** [+0.90, +6.11]	**+5.12** [+1.94, +6.74]	**+5.00** [+3.08, +6.97]
BoT-SORT	+0.28 [−1.43, +0.88]	+0.59† [−0.81, +1.29]	**+3.45** [+1.38, +5.53]
BoT-SORT + MASA ReID	+0.24† [−2.19, +1.29]	**+1.41** [+0.10, +2.28]	**+2.77** [+1.12, +4.50]
Deep OC-SORT + MASA	+0.51 [−0.89, +2.14]	**+2.59** [+0.04, +3.71]	**+2.37**† [+0.32, +4.59]
MASA	**+2.67** [+1.31, +6.30]	**+1.98** [+0.30, +2.82]	**+3.83** [+1.84, +5.55]
MASA-OA	**+3.43** [+0.95, +7.21]	**+0.60** [+0.08, +0.99]	**+8.43** [+5.24, +11.37]

Table 8: Per-sequence comparison of MASA-RF with each baseline: sequences won/lost on HOTA, exact two-sided sign-flip p-value on the mean per-sequence difference, and Holm-adjusted p-value over the seven baselines. Bold: adjusted $p < 0.05$. The smallest attainable adjusted p is 0.109 on MOT17-val and 0.875 on MOT20-val.

	MOT17-val ($n = 7$)			MOT20-val ($n = 4$)			DanceTrack-val ($n = 25$)		
Baseline	W/L	p	Holm	W/L	p	Holm	W/L	p	Holm
ByteTrack	7/0	0.016	0.109	2/2	0.750	1.000	25/0	<0.001	**<0.001**
OC-SORT	7/0	0.016	0.109	4/0	0.125	0.875	20/5	<0.001	**<0.001**
BoT-SORT	4/3	0.859	1.000	2/2	0.750	1.000	19/6	0.024	0.067
BoT-SORT + MASA ReID	2/5	0.469	1.000	4/0	0.125	0.875	17/8	0.018	0.067
Deep OC-SORT + MASA	5/2	0.438	1.000	3/1	0.250	1.000	15/10	0.066	0.067
MASA	7/0	0.016	0.109	3/1	0.250	1.000	19/6	0.017	0.067
MASA-OA	6/1	0.078	0.312	4/0	0.125	0.875	23/2	<0.001	**0.002**

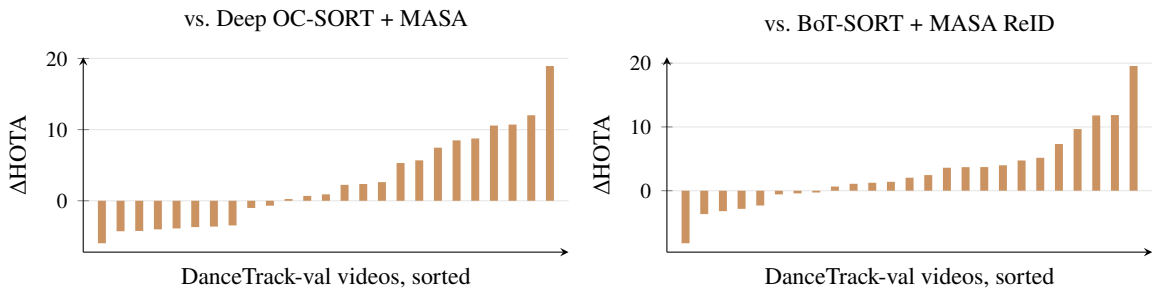


Figure 2: Per-sequence HOTA difference between MASA-RF and the two closest baselines on DanceTrack-val, sorted. Bars above zero are videos in which MASA-RF is better.

7.3. Robustness to configuration and post-processing

Tuning on the tuning split did not always help the baselines on the reporting splits: several published-default configurations are better than the tuned ones (Table 9), which is consistent with the weak rank correlation between tuning and reporting HOTA on DanceTrack (Section 8.1). As a conservative check that favours the baselines, we compare MASA-RF with the better of the tuned and default configurations of each baseline, selected with knowledge of the validation result. MASA-RF stays first on all splits, but its margin over the best baseline configuration shrinks to +0.09 on MOT17-val, +0.59 on MOT20-val and +1.73 on DanceTrack-val (BoT-SORT with MASA ReID, default parameters). In this comparison, the Holm-adjusted per-sequence test on DanceTrack-val remains significant only against ByteTrack, OC-SORT and MASA-OA (Table 10). With linear interpolation of gaps of up to 20 frames, MASA-RF also remains first on all three splits (Table 9).

Table 9: HOTA under alternative configurations and post-processing. T: tuned and locked configuration without interpolation (as in Table 6); D: published default parameters; I: tuned configuration with linear interpolation.

	MOT17-val			MOT20-val			DanceTrack-val		
Method	T	D	I	T	D	I	T	D	I
ByteTrack	67.20	67.81	67.98	58.80	58.30	59.39	46.42	47.25	46.30
OC-SORT	66.82	66.36	68.63	54.99	55.09	57.82	51.79	51.95	52.17
BoT-SORT	68.69	68.88	69.81	59.52	59.03	60.14	53.33	53.83	53.06
BoT-SORT + MASA ReID	68.73	68.51	69.88	58.70	58.76	59.25	54.01	55.06	53.87
Deep OC-SORT + MASA	68.46	67.51	69.91	57.52	55.36	60.36	54.42	53.34	53.91
MASA	66.29	65.69	66.75	58.13	54.63	60.19	52.96	49.44	52.65
MASA-OA	65.54	65.54	65.57	59.51	59.51	59.73	48.36	48.36	47.99
MASA-RF (ours)	68.97	–	70.16	60.11	–	60.71	56.79	–	56.58

Table 10: MASA-RF against the better of the tuned and default configurations of each baseline: combined HOTA difference, sequences won/lost, and Holm-adjusted sign-flip p-value. Bold: adjusted $p < 0.05$.

	MOT17-val			MOT20-val			DanceTrack-val		
Baseline	Δ	W/L	Holm	Δ	W/L	Holm	Δ	W/L	Holm
ByteTrack	+1.16	6/1	0.688	+1.31	2/2	1.000	+9.54	25/0	**<0.001**
OC-SORT	+2.15	7/0	0.109	+5.02	4/0	0.875	+4.83	20/5	**0.006**
BoT-SORT	+0.09	4/3	1.000	+0.59	2/2	1.000	+2.96	17/8	0.154
BoT-SORT + MASA ReID	+0.24	2/5	1.000	+1.35	4/0	0.875	+1.73	16/9	0.181
Deep OC-SORT + MASA	+0.51	5/2	1.000	+2.59	3/1	1.000	+2.37	15/10	0.154
MASA	+2.67	7/0	0.109	+1.98	3/1	1.000	+3.83	19/6	0.067
MASA-OA	+3.43	6/1	0.391	+0.60	4/0	0.875	+8.42	23/2	**0.003**

7.4. Contribution of each cue (post hoc for the pure cues)

Table 11 removes one component at a time from the locked configuration. Removing a cue requires care: on the MOT family, setting the constant weight to $w = 0$ does not remove appearance, because GAW and the cosine gate still use it, and $w = 1$ keeps the geometric gate. We therefore add *pure* variants that remove a cue from the cost and the gates of stages 1 and 3. Stage 2 always uses IoU.

On DanceTrack-val, appearance and geometry are complementary: pure geometry loses 5.07 HOTA and pure appearance 9.44. On MOT17-val, the pure variants lose 0.56 and 1.47, and on MOT20-val 0.04 and 1.74, so geometry carries most of the association on the MOT family and appearance adds little or nothing. The $w = 0$ row shows that on the MOT family, using appearance only where geometry is ambiguous is better than using it everywhere (+0.38 and +0.17 over $w = 0.3$). The low-score stage matters on the crowded MOT20-val (3.20 HOTA), and camera-motion compensation on MOT17-val (0.34 HOTA), where cameras move. ORM, which was selected on the DanceTrack tuning split, lowers HOTA on DanceTrack-val by 0.44; this difference is not significant (95% CI of HOTA with ORM minus HOTA without ORM: [−1.54, +0.70]), and the locked configuration is reported unchanged.

Fig. 3 sweeps the weight w (post hoc). HOTA decreases monotonically with w on the MOT family, while on DanceTrack-val it peaks at an intermediate value ($w = 0.5$: 57.20) and is more than 4 HOTA lower at both ends (51.72 at $w = 0$, 52.35 at $w = 1$).

Table 11: ΔHOTA when one component is removed from the locked MASA-RF configuration (negative: the component helps). "–": the component is not used in the locked configuration of that benchmark family.

Component removed	MOT17-val	MOT20-val	DanceTrack-val
Pure geometry: no appearance in the cost or gates	−0.56	−0.04	−5.07
Pure appearance: no IoU, centre term or geometric gate	−1.47	−1.74	−9.44
Constant weight $w = 0$ (GAW and cosine gate kept)	+0.38	+0.17	−5.07
Constant weight $w = 1$ (geometric gate kept)	−0.80	−1.48	−4.44
Centre term ($c = 0$)	–	–	−1.28
Geometry-ambiguity weighting ($\kappa = 0$)	−0.19	+0.10	–
Cosine gate	+0.15	−0.03	–
Camera-motion compensation	−0.34	−0.01	−0.07
Low-score stage (stage 2)	−0.16	−3.20	−0.07
Occlusion-risk memory (ORM)	–	–	+0.44

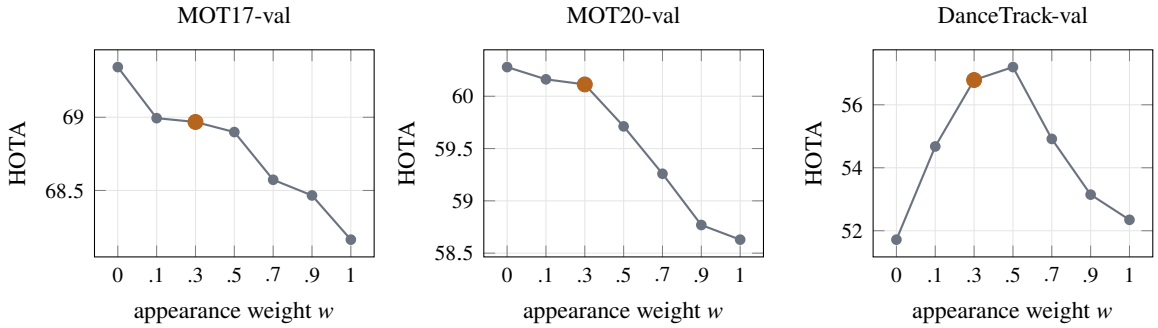


Figure 3: Post-hoc sensitivity of MASA-RF to the appearance weight w; other parameters as locked. On the MOT family, GAW and the cosine gate remain active at $w = 0$. Orange: locked value.

7.5. Appearance memory with real detections (post hoc)

The results of Section 4 were obtained with ground-truth boxes. To test memory protection with real detections, we vary only the memory of MASA-RF, with ORM off (Fig. 4 and Table 12). A memory that is never updated after the first observation ($\alpha = 0$) is the worst setting on every split (−2.42, −5.98 and −7.76 against $\alpha = 0.1$). Among non-zero rates, the differences are small and depend on the data: the range is 1.17 HOTA on MOT17-val, 0.50 on MOT20-val and 1.21 on DanceTrack-val, and the best rate is 0.05, 0.5 and 0.1, respectively. The search on the tuning splits selected $\alpha = 0.1$ on both families (Appendix A), but this sweep does not show that a slow memory is generally superior, only that an always-updated memory is much better than a fixed one.

We also apply a hard freeze, in the spirit of MASA-OA: $\alpha = 0$ whenever the occlusion risk satisfies $r_i \geq 0.3$. It lowers HOTA on all splits, for both a slow and a fast base rate, by 0.99–3.62 and 1.78–3.66, respectively (Table 12), and identity switches increase (with $\alpha = 0.1$: 184 to 264 on MOT17-val, 2282 to 3780 on MOT20-val, 1641 to 2041 on DanceTrack-val).

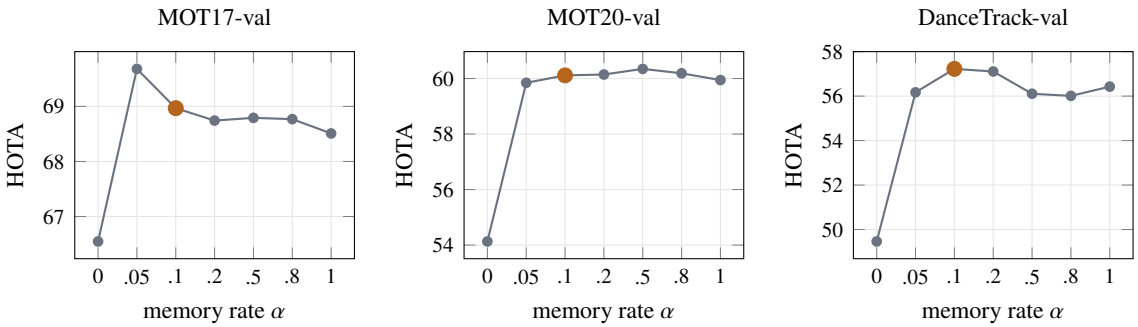


Figure 4: Post-hoc sensitivity of MASA-RF to the memory rate α, with ORM off and other parameters as locked. $\alpha = 0$ keeps the first observation. Orange: locked value.

Table 12: Post-hoc test of memory protection in MASA-RF with real detections (HOTA; ORM off). In parentheses: the change caused by the protection, relative to the same base rate without it ($\alpha = 0$ is compared with $\alpha = 0.1$).

Memory rule	MOT17-val	MOT20-val	DanceTrack-val
$\alpha = 0.1$, always updated	68.97	60.11	57.23
$\alpha = 0.1$, hard freeze when $r_i \geq 0.3$	67.97 (−0.99)	56.49 (−3.62)	54.28 (−2.94)
$\alpha = 0.8$, always updated	68.77	60.19	56.01
$\alpha = 0.8$, hard freeze when $r_i \geq 0.3$	66.96 (−1.81)	56.53 (−3.66)	54.23 (−1.78)
$\alpha = 0$: memory fixed at the first observation	66.55 (−2.42)	54.13 (−5.98)	49.46 (−7.76)

7.6. MASA-OA mechanisms with real detections (post hoc)

Table 13 repeats the mechanism ablation of Table 1 on detector output, with the locked MASA-OA configuration. Enforcing the freeze lowers HOTA on all splits (−0.43, −0.43 and −3.32), and the variant closest to the description in the paper (freeze, single Kalman correction and an overlap threshold of 0.3) is the lowest on all splits (−1.66, −2.18 and −4.18). The double Kalman correction has a small effect of inconsistent sign (+0.12, +0.26 and −0.37 when it is removed). Hence the finding from ground-truth boxes holds with real detections as well: enforcing the freeze does not improve the released tracker.

Table 13: Post-hoc mechanism ablation of the released MASA-OA tracker with real detections. Each row patches a copy of the author file, as in Table 1.

	MOT17-val			MOT20-val			DanceTrack-val		
Variant of the author code	HOTA	AssA	IDSW	HOTA	AssA	IDSW	HOTA	AssA	IDSW
Released code	65.54	66.69	300	59.51	56.18	2106	48.36	30.24	3028
Kalman correction applied once	65.66	66.77	305	59.77	56.65	2094	47.98	29.69	3110
Freeze enforced ($\alpha = 0$)	65.11	65.82	312	59.08	55.40	2308	45.04	26.31	3712
Freeze enforced + Kalman once	65.57	66.66	331	59.37	55.91	2275	44.97	26.22	3740
Overlap threshold IoU ≥ 0.3	65.45	66.70	269	59.01	55.36	2258	48.68	30.65	2972
Both points as described	63.88	63.60	369	57.32	52.32	3325	44.18	25.77	4792

7.7. Run time

Table 14 reports the association throughput of every method on identical cached inputs. Each method ran in its own process, one at a time, on an otherwise idle machine, with single-threaded numerical libraries; we report the median of three runs, and the spread between repetitions is below 10%. These runs also serve as a reproducibility check: in all 24 combinations of method and split, the tracker output is byte-identical to that of the main experiment.

Association cost grows with the number of objects per frame, so all methods are slowest on MOT20-val. ByteTrack is the fastest. Among appearance-based trackers, MASA-RF associates 1.5–1.6× faster than Deep OC-SORT and 3.3–8.8× faster than the released MASA-OA tracker, whose implementation updates tracks in per-detection Python loops. In a complete system the front end dominates (Table 15): the detector takes about 25 ms per frame, MASA-R50 about 58–61 ms and camera-motion estimation about 15 ms. The estimated total for MASA-RF is 98–111 ms per frame, of which association is less than 10%, against 26–30 ms for ByteTrack. Appearance-based trackers therefore pay mainly for the embedding network, not for association. Totals are sums of sequential components; they exclude image decoding, and a real system could run components in parallel. Fig. 5 shows accuracy against association speed.

Table 14: Run time. Each method uses the detector; the Emb. and CMC columns show whether it also needs the MASA embedding and camera motion. FPS and ms: association only, on a CPU with single-threaded numerics (median of three runs). Total: estimated time per frame of the complete pipeline, from the front-end costs of Table 15, without parallelism between components.

			MOT17-val			MOT20-val			DanceTrack-val		
Method	Emb.	CMC	FPS	ms	Total	FPS	ms	Total	FPS	ms	Total
ByteTrack			1068	0.9	26	194	5.2	30	2004	0.5	26
OC-SORT			953	1.0	26	190	5.3	30	1690	0.6	26
BoT-SORT		✓	951	1.1	41	179	5.6	45	1797	0.6	40
BoT-SORT + MASA ReID	✓	✓	774	1.3	99	129	7.7	108	1423	0.7	98
Deep OC-SORT + MASA	✓	✓	413	2.4	100	64	15.7	116	855	1.2	98
MASA	✓		724	1.4	84	78	12.8	99	1604	0.6	83
MASA-OA	✓		153	6.5	89	11	88.6	174	396	2.5	85
MASA-RF (ours)	✓	✓	639	1.6	99	99	10.1	111	1311	0.8	98

Table 15: Front-end cost per frame with one worker on the RTX 5070 Ti (camera motion on the CPU), measured on two videos of each split, excluding model warm-up.

Split	Frames	YOLOX-X (ms)	MASA-R50 (ms)	Camera motion (ms)
MOT17-val	780	24.9	57.6	15.1
MOT20-val	780	24.8	60.9	14.8
DanceTrack-val	780	25.0	57.7	14.4

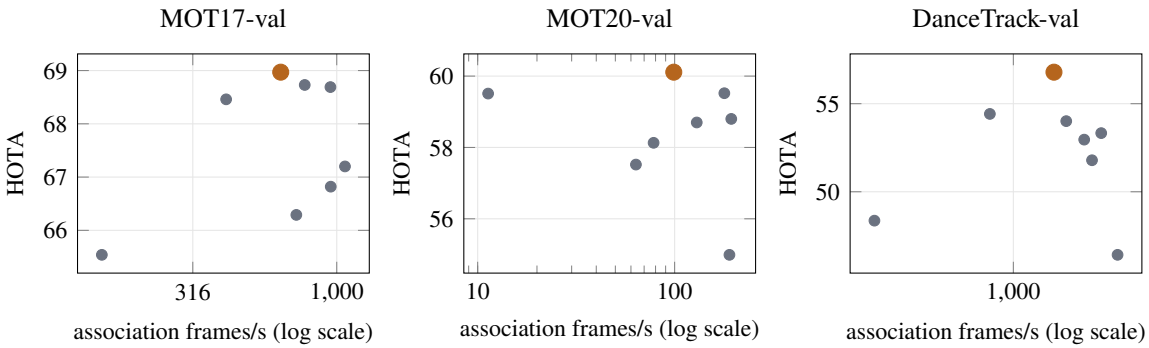


Figure 5: Accuracy versus association speed. Grey: the seven baselines of Table 14; orange: MASA-RF. The left-most grey point in every panel is the released MASA-OA tracker.

7.8. Qualitative example

Fig. 6 shows a crossing on DanceTrack-val in which Deep OC-SORT switches the identity of the tracked dancer while MASA-RF keeps it. The example was selected as the most crowded case among Deep OC-SORT switches in which MASA-RF keeps one identity for ±20 frames; it illustrates the failure mode and is not representative of all sequences.

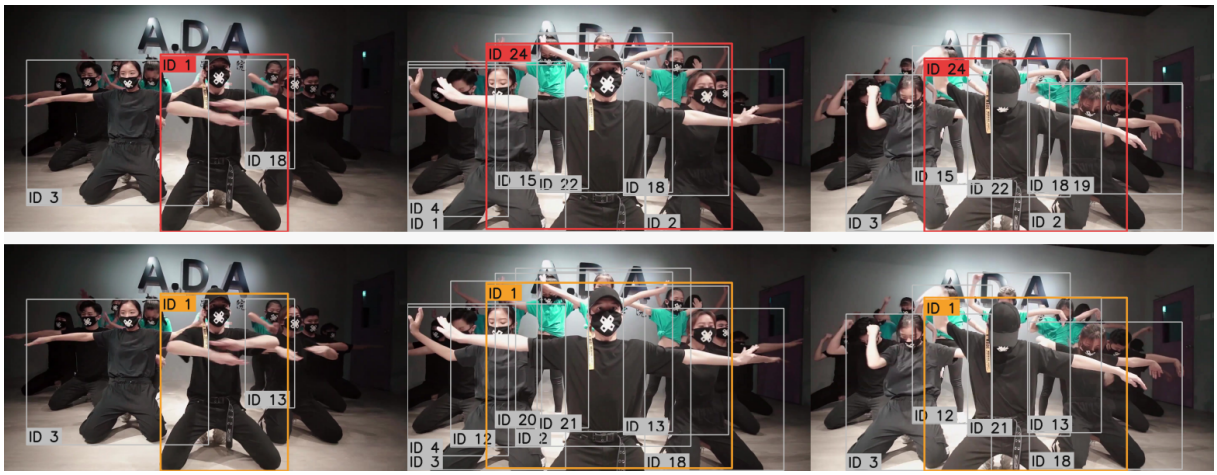


Figure 6: dancetrack0073, frames 251, 266 and 281. Top: Deep OC-SORT changes the identity of the highlighted dancer from 1 to 24. Bottom: MASA-RF keeps identity 1. Other boxes: other dancers.

8. Discussion

Why memory protection does not help. All experiments in which the memory is protected, whether with perfect or real boxes, in the authors' code or in ours, performed worse than an always-updated memory (Tables 1, 12 and 13). A plausible explanation, which we have not tested separately, is that appearance changes during occlusion and after it, through pose, scale and partial visibility; a memory that stops updating becomes stale, and the error this causes outweighs the contamination it prevents. Overlap is also only a proxy for contamination: many overlapping detections may have usable embeddings, and the freeze discards them.

How appearance and geometry should be combined. The best use of appearance depends on the data. With uniform clothing and non-linear motion (DanceTrack), neither cue is sufficient, and an intermediate weight is clearly better than either extreme (Fig. 3). With pedestrians that move smoothly (MOT family), geometry carries most of the association, and zero-shot appearance helps mainly where geometry is ambiguous. This explains why a single simple cost can match or exceed more elaborate trackers when all share the same inputs, and it agrees with earlier findings on simple cues [24].

Implications for evaluation. Our study suggests three practices for association research. First, ground-truth boxes should not be the only protocol, since they saturate HOTA and reward geometry (Table 2). Second, new mechanisms should be verified by ablation in the code that produced the results, because a mechanism described in a paper may not be active in the released implementation. Third, differences should be tested per sequence and with multiplicity correction: on benchmarks with few sequences such as MOT20-val, small differences cannot be confirmed statistically, however consistent they appear.

8.1. Threats to validity

Internal validity. The box-clipping convention for MOT20 and DanceTrack was fixed after a first validation run, on the basis of ground-truth statistics. All experiments labelled post hoc were designed after the validation results were known; we use them to explain results and not to change any configuration, and values above the locked result in those experiments (for example $\alpha = 0.05$ on MOT17-val) are not claims about achievable performance. Search budgets are not equal: small search spaces were searched exhaustively, and MASA-RF received fewer configurations than Deep OC-SORT and BoT-SORT with MASA ReID but more than the motion-only baselines. The comparison with the better of the tuned and default configurations partly addresses this. The rank correlation between tuning and reporting HOTA across methods is 0.87 on MOT17 but only 0.42 on DanceTrack, so the DanceTrack tuning split is a poor proxy for validation; this led to the selection of ORM.

External validity. All numbers are on validation splits; test-set results have not been obtained. MOT20-val uses a detector trained on another benchmark, and its configurations were tuned on MOT17. All appearance-based trackers use the zero-shot MASA embedding, and each benchmark uses one detector. Deep OC-SORT with its own trained ReID model reports 61.3 HOTA on the DanceTrack test set in its official repository; this is not directly comparable with our validation numbers, but it indicates that trained ReID features can outperform the zero-shot setting. Our conclusions therefore hold for identical, training-free appearance features.

Statistical conclusion validity. The number of sequences limits every test, and sequences within a benchmark are not independent samples of all tracking scenarios. The bootstrap combines HOTA over sequences, whereas the permutation test uses the mean per-sequence difference, so the two can differ when sequences vary in length. We report both for this reason.

9. Conclusion

The improvement of MASA-OA does not come from freezing the appearance memory: the freeze is inert in the released code, and enforcing it lowers HOTA with both ground-truth boxes and real detections. The ground-truth-box protocol also cannot measure the contribution of appearance. Under a controlled detector-based protocol, a simple training-free fusion of MASA appearance with geometry in a three-stage cascade, with an always-updated memory, reaches the highest HOTA on MOT17-val, MOT20-val and DanceTrack-val. Its gain is statistically supported on DanceTrack-val, firmly against ByteTrack, OC-SORT and MASA-OA, and less firmly against the strongest appearance-based baselines; on MOT17-val and MOT20-val it is on a par with the strongest baselines. Future work will evaluate the locked configurations on the test servers, add StrongSORT [4], Hybrid-SORT [15] and GHOST [24] with the same inputs, study the sensitivity of the conclusions to the detector, and test directly whether embedding quality degrades under occlusion in a way that a selective update could exploit.

Appendix A. Search spaces and locked configurations

Table A.16: Search spaces of the first tuning round; later rounds extend the boundaries and refine these ranges (scripts in the accompanying code). Parameter names are those of the official implementations; MASA-RF names are defined in the accompanying code.

Method	Search space
ByteTrack	track_thresh ∈ {0.5, 0.6, 0.7}; match_thresh ∈ {0.7, 0.8, 0.9}; track_buffer ∈ {30, 60}
OC-SORT	det_thresh ∈ {0.5, 0.6, 0.7}; iou_threshold ∈ {0.2, 0.3}; inertia ∈ {0.1, 0.2}; use_byte ∈ {False, True}
BoT-SORT	track_high_thresh ∈ {0.5, 0.6}; new_track_thresh ∈ {0.6, 0.7}; match_thresh ∈ {0.7, 0.8}; track_buffer ∈ {30, 60}
BoT-SORT + MASA ReID	appearance_thresh ∈ {0.15, 0.2, 0.25, 0.3, 0.35}; proximity_thresh ∈ {0.4, 0.5, 0.6}; match_thresh ∈ {0.7, 0.8}
Deep OC-SORT + MASA	w_association_emb ∈ {0.5, 0.75, 1.0, 1.5}; alpha_fixed_emb ∈ {0.9, 0.95}; det_thresh ∈ {0.5, 0.6, 0.7}; aw_param ∈ {0.5, 1.0}
MASA	match_threshold ∈ {0.3, 0.5}; init_score_threshold ∈ {0.6, 0.8}; object_score_threshold ∈ {0.3, 0.5}; max_age ∈ {10, 30}
MASA-OA	match_score_thr ∈ {0.3, 0.5}; init_score_thr ∈ {0.6, 0.8}; obj_score_thr ∈ {0.3, 0.5}; memo_tracklet_frames ∈ {10, 30}
MASA-RF	appearance_weight ∈ {0.3, 0.5, 0.7}; center_weight ∈ {0.0, 0.25, 0.5}; ema_alpha ∈ {0.1, 0.2, 0.5, 0.8}; risk_memory ∈ {0.0, 0.5, 0.9}; adaptive_weight ∈ {0.0, 0.5, 1.0}; match_thresh ∈ {0.7, 0.8, 0.9}; appearance_mode ∈ {masa, cosine}; age_penalty ∈ {0.0, 0.02, 0.05}

Table A.17: Locked configurations and HOTA on the tuning split. Unlisted parameters keep their defaults.

Method	Tuning HOTA	Parameters
MOT17/MOT20		
ByteTrack	75.50	`track_thresh=0.5, match_thresh=0.9, track_buffer=30`
OC-SORT	75.87	`det_thresh=0.6, iou_threshold=0.2, inertia=0.1, use_byte=False`
BoT-SORT	77.96	`track_high_thresh=0.6, new_track_thresh=0.6, match_thresh=0.8, track_buffer=30`
BoT-SORT + MASA ReID	78.74	`appearance_thresh=0.3, proximity_thresh=0.5, match_thresh=0.8`
Deep OC-SORT + MASA	78.20	`w_association_emb=1.5, alpha_fixed_emb=0.8, det_thresh=0.6, aw_param=0.5, max_age=60, min_hits=1, iou_threshold=0.2, inertia=0.1, delta_t=3`
MASA	77.30	`match_threshold=0.5, init_score_threshold=0.6, object_score_threshold=0.5, max_age=30`
MASA-OA	74.00	`match_score_thr=0.5, init_score_thr=0.8, obj_score_thr=0.5, memo_tracklet_frames=10`
MASA-RF	78.66	`appearance_weight=0.3, center_weight=0.0, ema_alpha=0.1, risk_memory=0.0, adaptive_weight=1.0, match_thresh=0.7, appearance_mode=masa, age_penalty=0.0, new_track_thresh=0.6, high_thresh=0.6, track_buffer=30, appearance_gate=0.5, geometry_gate=0.15, fuse_score=True`
DanceTrack		
ByteTrack	54.94	`track_thresh=0.5, match_thresh=0.8, track_buffer=30`
OC-SORT	64.47	`det_thresh=0.5, iou_threshold=0.2, inertia=0.1, use_byte=False`
BoT-SORT	60.09	`track_high_thresh=0.5, new_track_thresh=0.7, match_thresh=0.8, track_buffer=30`
BoT-SORT + MASA ReID	59.34	`appearance_thresh=0.4, proximity_thresh=0.5, match_thresh=0.7, new_track_thresh=0.6, track_high_thresh=0.6, track_buffer=60, track_low_thresh=0.1`
Deep OC-SORT + MASA	68.17	`w_association_emb=1.0, alpha_fixed_emb=0.95, det_thresh=0.5, aw_param=0.5, max_age=30, min_hits=1, iou_threshold=0.3, inertia=0.2, delta_t=3`
MASA	57.25	`match_threshold=0.3, init_score_threshold=0.8, object_score_threshold=0.3, max_age=10`
MASA-OA	61.03	`match_score_thr=0.3, init_score_thr=0.8, obj_score_thr=0.3, memo_tracklet_frames=10`
MASA-RF	62.40	`appearance_weight=0.3, center_weight=0.25, ema_alpha=0.1, risk_memory=0.9, adaptive_weight=0.0, match_thresh=0.9, appearance_mode=masa, age_penalty=0.0`

Appendix B. Complete metrics

Table B.18: Complete TrackEval metrics on the reporting splits (raw output).

Method	HOTA	DetA	AssA	AssRe	AssPr	LocA	IDF1	MOTA	IDSW	Frag	FP	FN
MOT17-val												
ByteTrack	67.20	66.34	68.67	74.59	81.09	86.18	79.18	77.66	237	512	3353	8447
OC-SORT	66.82	64.30	69.87	74.43	84.68	87.27	78.29	74.69	226	585	1299	12112
BoT-SORT	68.69	67.16	70.81	76.05	82.72	86.53	80.87	78.19	200	517	3258	8295
BoT-SORT + MASA ReID	68.73	67.30	70.71	75.75	83.19	86.69	81.50	78.48	175	453	2713	8708
Deep OC-SORT + MASA	68.46	66.14	71.33	77.12	82.70	86.93	80.90	77.28	158	868	1935	10153
MASA	66.29	66.62	66.51	71.63	83.16	86.76	76.85	77.29	275	879	2662	9301
MASA-OA	65.54	64.97	66.69	72.61	80.55	86.42	75.48	74.68	300	522	3825	9520
MASA-RF	68.97	67.02	71.49	75.91	85.01	86.55	81.69	78.26	184	504	2954	8579
MOT20-val												
ByteTrack	58.80	63.73	54.44	60.43	72.53	82.95	73.83	80.58	2639	6794	23596	194133
OC-SORT	54.99	56.56	53.61	57.65	77.72	83.99	70.64	69.98	2065	13742	7873	330713
BoT-SORT	59.52	63.96	55.60	60.77	75.48	83.38	74.22	80.31	3442	6584	19430	200550
BoT-SORT + MASA ReID	58.70	63.28	54.64	59.95	74.55	83.45	73.24	79.42	2605	5710	17533	213343
Deep OC-SORT + MASA	57.52	59.27	55.99	60.80	76.62	83.81	73.89	73.65	2347	25105	9899	286672
MASA	58.13	61.78	54.88	58.62	79.72	83.62	73.10	76.90	2832	21853	13519	245690
MASA-OA	59.51	63.25	56.18	61.98	74.23	83.11	75.16	79.55	2106	8006	26153	203751
MASA-RF	60.11	64.17	56.51	60.23	79.88	83.41	74.93	80.22	2282	5747	18626	203471
DanceTrack-val												
ByteTrack	46.42	70.90	30.52	36.26	58.58	84.54	50.86	88.29	2084	3534	8749	15532
OC-SORT	51.79	78.00	34.50	39.96	65.47	91.45	51.45	88.02	2159	3846	2964	21851
BoT-SORT	53.33	78.36	36.45	42.90	63.37	89.85	54.06	90.31	1931	2873	6815	13065
BoT-SORT + MASA ReID	54.01	78.78	37.18	42.44	66.70	89.90	54.02	90.09	2217	2925	6817	13275
Deep OC-SORT + MASA	54.42	79.49	37.39	42.45	69.71	91.18	53.40	89.47	2553	3884	6037	15125
MASA	52.96	79.21	35.59	39.78	68.92	91.19	50.45	89.25	2319	3509	6026	15862
MASA-OA	48.36	77.78	30.24	34.28	67.37	90.80	44.42	87.74	3028	3172	12816	11760
MASA-RF	56.79	79.14	40.89	47.44	64.42	91.02	58.29	90.27	1710	3137	5716	14484

Table B.19: Per-sequence HOTA on the reporting splits. Best value per row in bold. BoT-R: BoT-SORT + MASA ReID; DOCS: Deep OC-SORT + MASA; RF: MASA-RF; dt: dancetrack.

Sequence	ByteTrack	OC-SORT	BoT-SORT	BoT-R	DOCS	MASA	MASA-OA	RF
MOT17-02	46.2	44.2	46.9	**48.3**	45.5	39.4	40.0	47.9
MOT17-04	79.0	79.4	79.3	78.3	79.5	79.0	78.3	**80.0**
MOT17-05	60.8	57.7	63.9	64.7	60.2	57.6	57.5	**65.3**
MOT17-09	64.7	62.6	**67.0**	66.7	64.0	60.9	64.7	65.6
MOT17-10	55.2	54.1	57.4	**59.0**	58.5	55.8	52.0	58.5
MOT17-11	61.7	62.9	67.2	**67.9**	67.2	60.9	67.1	63.1
MOT17-13	64.5	59.2	69.4	**69.8**	68.5	67.8	63.4	69.1
MOT20-01	**65.4**	61.1	64.2	64.1	62.8	62.0	62.4	64.5
MOT20-02	58.3	56.3	**59.0**	57.4	58.3	58.1	58.0	58.1
MOT20-03	61.1	58.3	**62.0**	61.8	60.2	60.7	61.8	61.9
MOT20-05	57.2	52.3	57.9	56.9	55.4	56.4	58.3	**59.2**
dt0004	38.9	42.4	46.9	**60.5**	54.4	45.4	43.6	60.0
dt0005	67.2	**72.0**	52.1	63.5	51.9	54.5	52.6	70.8
dt0007	43.4	49.0	50.3	49.3	49.9	45.6	**51.6**	45.6
dt0010	48.4	61.9	63.0	66.7	58.3	**75.8**	65.0	70.3
dt0014	38.1	41.2	**44.1**	40.1	43.4	37.5	39.3	39.5
dt0018	79.4	79.9	78.2	67.3	76.3	86.8	67.7	**86.8**
dt0019	31.4	37.2	38.3	34.8	39.3	38.9	32.8	**39.6**
dt0025	49.2	60.8	60.0	**66.4**	63.1	59.6	44.9	64.1
dt0026	44.7	**52.7**	48.7	48.9	48.3	47.5	49.2	50.9
dt0030	67.9	76.9	88.4	86.4	82.6	82.5	79.1	**90.1**
dt0034	45.0	44.7	46.6	44.9	**49.3**	43.5	45.4	48.6
dt0035	37.5	42.0	45.3	41.8	**49.5**	42.2	37.9	45.8
dt0041	35.0	34.7	36.8	40.1	40.6	32.3	35.7	**41.3**
dt0043	52.9	49.2	55.1	56.5	**57.2**	56.5	47.8	53.6
dt0047	43.3	45.8	51.5	45.5	46.4	50.9	41.0	**55.1**
dt0058	45.9	52.0	52.3	55.2	58.6	62.6	46.9	**67.1**
dt0063	29.2	27.4	29.1	33.4	29.5	23.8	22.6	**34.8**
dt0065	46.4	57.8	65.0	52.5	**67.7**	60.4	38.5	64.3
dt0073	50.8	49.8	51.9	56.3	**57.1**	53.1	44.7	53.1
dt0077	60.7	62.8	68.3	70.0	59.9	64.0	52.5	**70.6**
dt0079	57.9	68.8	67.9	74.5	**78.0**	73.4	70.2	77.0
dt0081	44.3	52.6	51.6	48.4	51.2	45.0	46.6	**53.5**
dt0090	39.9	46.6	49.4	49.4	**53.4**	49.3	38.4	49.1
dt0094	46.7	52.0	53.7	56.4	55.2	52.7	52.0	**57.5**
dt0097	42.4	50.5	60.0	53.5	51.3	64.1	**71.6**	45.3

References

[1] H. W. Kuhn, The Hungarian method for the assignment problem, Nav. Res. Logist. Q. 2 (1–2) (1955) 83–97. doi:10.1002/nav.3800020109.

[2] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, X. Wang, ByteTrack: Multi-object tracking by associating every detection box, in: Computer Vision – ECCV 2022, Vol. 13682 of Lecture Notes in Computer Science, Springer Nature Switzerland, Cham, 2022, pp. 1–21. doi:10.1007/978-3-031-20047-2_1.

[3] J. Cao, J. Pang, X. Weng, R. Khirodkar, K. Kitani, Observation-centric SORT: Rethinking SORT for robust multi-object tracking, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 9686–9696. doi:10.1109/CVPR52729.2023.00934.

[4] Y. Du, Z. Zhao, Y. Song, Y. Zhao, F. Su, T. Gong, H. Meng, StrongSORT: Make DeepSORT great again, IEEE Trans. Multimed. 25 (2023) 8725–8737. doi:10.1109/TMM.2023.3240881.

[5] M. Ye, J. Shen, G. Lin, T. Xiang, L. Shao, S. C. H. Hoi, Deep learning for person re-identification: A survey and outlook, IEEE Trans. Pattern Anal. Mach. Intell. 44 (6) (2022) 2872–2893. doi:10.1109/TPAMI.2021.3054775.

[6] P. Sun, J. Cao, Y. Jiang, Z. Yuan, S. Bai, K. Kitani, P. Luo, DanceTrack: Multi-object tracking in uniform appearance and diverse motion, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 20961–20970. doi:10.1109/CVPR52688.2022.02032.

[7] S. Li, L. Ke, M. Danelljan, L. Piccinelli, M. Segu, L. Van Gool, F. Yu, Matching anything by segmenting anything, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 18963–18973. doi:10.1109/CVPR52733.2024.01794.

[8] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, R. Girshick, Segment anything, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 3992–4003. doi:10.1109/ICCV51070.2023.00371.

[9] J. Pang, L. Qiu, X. Li, H. Chen, Q. Li, T. Darrell, F. Yu, Quasi-dense similarity learning for multiple object tracking, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2021, pp. 164–173. doi:10.1109/CVPR46437.2021.00023.

[10] T. Fischer, T. E. Huang, J. Pang, L. Qiu, H. Chen, T. Darrell, F. Yu, QDTrack: Quasi-dense similarity learning for appearance-only multiple object tracking, IEEE Trans. Pattern Anal. Mach. Intell. 45 (12) (2023) 15380–15393. doi:10.1109/TPAMI.2023.3301975.

[11] G. Carretero, J. P. Llenera, L. Usero, M. A. Patricio, Multiple object tracker with integrated embedding-based optimization and occlusion-aware variants, Inf. Sci. 749 (2026) 123533. doi:10.1016/j.ins.2026.123533.

[12] K. Musgrave, S. Belongie, S.-N. Lim, A metric learning reality check, in: Computer Vision – ECCV 2020, Vol. 12370 of Lecture Notes in Computer Science, Springer International Publishing, Cham, 2020, pp. 681–699. doi:10.1007/978-3-030-58595-2_41.

[13] P. Bergmann, T. Meinhardt, L. Leal-Taixé, Tracking without bells and whistles, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2019, pp. 941–951. doi:10.1109/ICCV.2019.00103.

[14] X. Zhou, V. Koltun, P. Krähenbühl, Tracking objects as points, in: Computer Vision – ECCV 2020, Vol. 12349 of Lecture Notes in Computer Science, Springer International Publishing, Cham, 2020, pp. 474–490. doi:10.1007/978-3-030-58548-8_28.

[15] M. Yang, G. Han, B. Yan, W. Zhang, J. Qi, H. Lu, D. Wang, Hybrid-SORT: Weak cues matter for online multi-object tracking, Proc. AAAI Conf. Artif. Intell. 38 (7) (2024) 6504–6512. doi:10.1609/aaai.v38i7.28471.

[16] K. Yi, K. Luo, X. Luo, J. Huang, H. Wu, R. Hu, W. Hao, UCMCTrack: Multi-object tracking with uniform camera motion compensation, Proc. AAAI Conf. Artif. Intell. 38 (7) (2024) 6702–6710. doi:10.1609/aaai.v38i7.28493.

[17] Z. Qin, S. Zhou, L. Wang, J. Duan, G. Hua, W. Tang, MotionTrack: Learning robust short-term and long-term motions for multi-object tracking, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 17939–17948. doi:10.1109/CVPR52729.2023.01720.

[18] W. Lv, Y. Huang, N. Zhang, R.-S. Lin, M. Han, D. Zeng, DiffMOT: A real-time diffusion-based multiple object tracker with non-linear prediction, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 19321–19330. doi:10.1109/CVPR52733.2024.01828.

[19] J. Shi, C. Tomasi, Good features to track, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 1994, pp. 593–600. doi:10.1109/CVPR.1994.323794.

[20] B. D. Lucas, T. Kanade, An iterative image registration technique with an application to stereo vision, in: Proceedings of the 7th International Joint Conference on Artificial Intelligence (IJCAI), Vol. 2, Vancouver, Canada, 1981, pp. 674–679.

[21] M. A. Fischler, R. C. Bolles, Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography, Commun. ACM 24 (6) (1981) 381–395. doi:10.1145/358669.358692.

[22] Z. Wang, L. Zheng, Y. Liu, Y. Li, S. Wang, Towards real-time multi-object tracking, in: Computer Vision – ECCV 2020, Vol. 12356 of Lecture Notes in Computer Science, Springer International Publishing, Cham, 2020, pp. 107–122. doi:10.1007/978-3-030-58621-8_7.

[23] Y. Zhang, C. Wang, X. Wang, W. Zeng, W. Liu, FairMOT: On the fairness of detection and re-identification in multiple object tracking, Int. J. Comput. Vis. 129 (11) (2021) 3069–3087. doi:10.1007/s11263-021-01513-4.

[24] J. Seidenschwarz, G. Brasó, V. C. Serrano, I. Elezi, L. Leal-Taixé, Simple cues lead to a strong multi-object tracker, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 13813–13823. doi:10.1109/CVPR52729.2023.01327.

[25] Y.-H. Wang, J.-W. Hsieh, P.-Y. Chen, M.-C. Chang, H.-H. So, X. Li, SMILEtrack: SiMIlarity LEarning for occlusion-aware multiple object tracking, Proc. AAAI Conf. Artif. Intell. 38 (6) (2024) 5740–5748. doi:10.1609/aaai.v38i6.28386.

[26] S. Li, M. Danelljan, H. Ding, T. E. Huang, F. Yu, Tracking every thing in the wild, in: Computer Vision – ECCV 2022, Vol. 13682 of Lecture Notes in Computer Science, Springer Nature Switzerland, Cham, 2022, pp. 498–515. doi:10.1007/978-3-031-20047-2_29.

[27] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778. doi:10.1109/CVPR.2016.90.

[28] G. Brasó, L. Leal-Taixé, Learning a neural solver for multiple object tracking, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 6246–6256. doi:10.1109/CVPR42600.2020.00628.

[29] O. Cetintas, G. Brasó, L. Leal-Taixé, Unifying short and long-term tracking with graph hierarchies, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 22877–22887. doi:10.1109/CVPR52729.2023.02191.

[30] T. Meinhardt, A. Kirillov, L. Leal-Taixé, C. Feichtenhofer, TrackFormer: Multi-object tracking with transformers, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 8834–8844. doi:10.1109/CVPR52688.2022.00864.

[31] F. Zeng, B. Dong, Y. Zhang, T. Wang, X. Zhang, Y. Wei, MOTR: End-to-end multiple-object tracking with transformer, in: Computer Vision – ECCV 2022, Vol. 13687 of Lecture Notes in Computer Science, Springer Nature Switzerland, Cham, 2022, pp. 659–675. doi:10.1007/978-3-031-19812-0_38.

[32] Y. Zhang, T. Wang, X. Zhang, MOTRv2: Bootstrapping end-to-end multi-object tracking by pre-trained object detectors, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 22056–22065. doi:10.1109/CVPR52729.2023.02112.

[33] R. Gao, L. Wang, MeMOTR: Long-term memory-augmented transformer for multi-object tracking, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 9867–9876. doi:10.1109/ICCV51070.2023.00908.

[34] P. Dendorfer, A. Ošep, A. Milan, K. Schindler, D. Cremers, I. Reid, S. Roth, L. Leal-Taixé, MOTChallenge: A benchmark for single-camera multiple target tracking, Int. J. Comput. Vis. 129 (4) (2021) 845–881. doi:10.1007/s11263-020-01393-0.

[35] J. Luiten, A. Ošep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, B. Leibe, HOTA: A higher order metric for evaluating multi-object tracking, Int. J. Comput. Vis. 129 (2) (2021) 548–578. doi:10.1007/s11263-020-01375-2.

[36] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché-Buc, E. Fox, H. Larochelle, Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program), J. Mach. Learn. Res. 22 (164) (2021) 1–20.
URL https://jmlr.org/papers/v22/20-303.html

[37] J. Demšar, Statistical comparisons of classifiers over multiple data sets, J. Mach. Learn. Res. 7 (1) (2006) 1–30.
URL https://www.jmlr.org/papers/v7/demsar06a.html

[38] B. Efron, Bootstrap methods: Another look at the jackknife, Ann. Stat. 7 (1) (1979) 1–26. doi:10.1214/aos/1176344552.

[39] S. Holm, A simple sequentially rejective multiple test procedure, Scand. J. Stat. 6 (2) (1979) 65–70.
URL http://www.jstor.org/stable/4615733